The Broadest Reasonable Prompt

Patent Claiming, Adversarial Interpretation, and the Lawyer's Place in Natural-Language Programming

Watson Hartsoe

Working Paper · August 17, 2026

"Except insofar as you're thinking adversarial, you're thinking, Okay, well, how is this gonna be screwed up?" — Shambibble, patent attorney and Midjourney practitioner, October 22, 2022.

AI-augmented working paper
Research and assembly record: Appendix A and accompanying SOURCE_MAP.

Abstract

In October 2022, an unusually clear early theory of prompting came not from computer science but from a patent attorney using the name Shambibble. Patent practice, he explained, had given him a "facility with describing a thing" in abstract language likely to be misinterpreted. The transferable habit was adversarial: ask how the language will be "screwed up." This Article takes that observation seriously. Patent law has spent more than a century building institutions around the same problem: operative language must allocate scope under interpretation by actors the drafter does not control. Claims receive constructions that may exceed private intention; definiteness demands reasonable certainty rather than impossible perfection; the specification disciplines meaning; open and closed transitional phrases alter scope; antecedent basis stabilizes reference; and amendments leave a prosecution history that constrains what can later be claimed. These are not merely analogies to clever prompting. They describe a mature craft of interpretive engineering. I argue that the lawyer's potential leadership in prompt language lies not in legalese or verbosity but in drafting for adverse construction, managing residual ambiguity, preserving amendment history, and knowing when language has become certain enough to authorize consequence. The Article proposes a Broadest Reasonable Prompt test and develops a deeper claim: the mature unit of prompt practice may resemble patent prosecution more than source code. A prompt is proposed, construed, rejected by behavior, amended, and tested again. The model is not a court, and patent doctrine is not computational semantics. That difference matters. It makes Shambibble's second discipline—empirical testing under stochastic execution—the necessary complement to lawyerly construction.

Keywords: prompt language; patent law; claim construction; adversarial interpretation; natural-language programming; generative AI; legal drafting; specification

I. An Ancient Master Appears in PromptCraft

In October 2022, before "prompt engineer" had stabilized as an occupational label, Watson Hartsoe interviewed a Midjourney user known as Shambibble. Asked whether his professional background explained his unusual facility with prompting, Shambibble answered that he was a patent attorney. The relevant transfer, he said, was not copyright doctrine and not artistic training. It was the "facility with describing a thing" using abstract language that was "very likely to be misinterpreted" if it ever reached court.[1]

That answer deserves to be read as more than a biographical curiosity. When Hartsoe suggested that patent practice had become a "creative superpower," Shambibble resisted the romance. The skill, he explained, was thinking hard about double meanings and ways words could be misread. His toy example was "orange." A user may want the color while the model produces the fruit. The repair is adversarial drafting: "no fruit, no sphere, no circle." Then comes the line that should be treated as an early jurisprudence of prompt language: "you're thinking adversarial, you're thinking, Okay, well, how is this gonna be screwed up?"[2]

Calling Shambibble an ancient master is useful only if the phrase is stripped of mystique. He was not necessarily the first prompt practitioner, and this Article makes no claim of priority. He is "ancient" in a more consequential sense: the discipline he imports into prompting is older than prompting. Patent lawyers had already spent generations learning to write operative language for an interpreter they do not control, under conditions where private intention cannot rescue public text after the fact. Prompting suddenly made that old craft computationally visible.

[1] *MJ Interview 3.wh_shambibble*, interview transcript, Oct. 22, 2022, at approximately 02:10–04:01 (automated transcript on file with author).

[2] Id.

The conventional story is that lawyers write precisely. That is too shallow. Patent law's own doctrine denies that absolute precision is possible. The deeper professional competence is *interpretive engineering*: identifying which ambiguities matter, anticipating adverse constructions, defining scope, recording amendments, and leaving enough uncertainty for the instrument to remain usable without leaving enough to make its boundary unknowable.

II. The Interpreter Is Not Cooperative

Most everyday language relies on cooperative inference. Speakers omit what listeners can safely recover; references remain loose; examples stand in for rules; intention repairs imperfect phrasing. Patent claims are different because interpretation occurs under consequence. The drafter must assume that another actor—examiner, competitor, judge, jury, or later court—may read the language against the drafter's preferred result.

That adversarial stance has a doctrinal form. During examination, the Patent and Trademark Office gives pending claims their broadest reasonable interpretation consistent with the specification.[3] The point is not that every imaginable reading wins. The point is that the applicant does not receive a monopoly on the intended reading. The claim is tested from outside.

Shambibble describes the same practical reversal. When "orange" yields fruit, the relevant fact is not that a reasonable human friend would have understood the color. The machine has demonstrated an available construction. The prompt writer either accepts that construction as harmless or changes the text. This is why his patent analogy is stronger than an analogy to technical writing. Technical writing usually imagines successful transmission to a cooperative reader. Patent practice institutionalizes the possibility of hostile construction.

The Supreme Court's nineteenth-century warning in *White v. Dunbar* makes the point in memorable form: a patent claim cannot be treated "like a nose of wax" and twisted after the fact toward whatever construction happens to be convenient.[4] Prompt practice has its own nose-of-wax temptation. After an unwanted output, the user says: *but that is obviously not what I meant.* The claim-drafting response is harsher and more productive: if the unwanted construction was available in the operative language, preserve the failure and amend the instrument.

That shift turns failure into evidence about scope. The model's output is not authoritative interpretation in the legal sense; it can hallucinate, ignore instructions, or behave erratically. But where an output instantiates a textually plausible reading, the event exposes a drafting ambiguity that private intention had concealed. An advanced prompt practice should therefore distinguish two questions that are often collapsed: *Did the model comply with the prompt?* and *Did the prompt exclude the behavior the user did not want?*

III. The Broadest Reasonable Prompt

The first operational proposal of this Article is a *Broadest Reasonable Prompt* test. Before consequential execution, construe the instruction not only as the user intends it, but through the materially different readings an agent could reasonably take in light of its context and runtime.

The word *reasonable* is doing hard work. It prevents adversarial drafting from degenerating into enumeration of every logical possibility. Patent law similarly rejects both perfect certainty and

[3] U.S. Patent & Trademark Office, *Manual of Patent Examining Procedure* § 2111 (current ed.), `https://www.uspto.gov/web/offices/pac/mpep/s2111.html`.

[4] 119 U.S. 47, 51–52 (1886), `https://www.govinfo.gov/content/pkg/USREPORTS-119/pdf/USREPORTS-119-47.pdf`.

boundless ambiguity. Section 112(b) requires claims that "particularly pointing out and distinctly claiming" the invention.[5] In *Nautilus v. Biosig*, the Court held that claims, read with the specification and prosecution history, must inform skilled readers of scope with "reasonable certainty," while recognizing that "absolute precision is unattainable."[6]

For prompt language, this suggests a standard more useful than "be as specific as possible." A prompt is reasonably certain when its surviving ambiguity cannot move execution across a material boundary the user cares about. A creative prompt may intentionally leave color, composition, or wording open. A file-deletion instruction cannot safely leave the target repository open. Precision should concentrate around consequential scope.

A Broadest Reasonable Prompt review therefore asks at least six questions. First, which nouns can refer to more than one live entity? Second, which verbs leave unclear whether the agent should advise, prepare, or act? Third, which lists are open to additions and which are exhaustive? Fourth, which relative terms—"small," "appropriate," "fast," "clean," "safe"—lack a standard for application? Fifth, which examples might be read as requirements, and which requirements might be mistaken for examples? Sixth, which prior constraints remain active even though the current text no longer mentions them?

The USPTO's examination forms are unexpectedly suggestive here. They treat phrases such as "for example" and "such as" as possible sources of indefiniteness when it is unclear whether what follows is part of the claimed limitation, and they flag failures of antecedent basis where expressions such as "the lever" have not been adequately introduced.[7] These can look like lawyerly fussiness until an autonomous agent has five open files, three versions of a character, two customer records with the same surname, and authority to send or delete. Then antecedent basis begins to look like state safety.

IV. Scope Is an Operator

Patent claims also contain a feature prompt language has barely begun to formalize: ordinary-looking words with institutionally stabilized scope effects. "Comprising" is presumptively open-ended; it requires the recited elements without excluding additional compatible elements. "Consisting of" operates as a closed transition.[8]

This distinction is profound for agentic language. Consider "make the report include revenue, retention, and churn." Is that an open set, permitting a model to add margin, acquisition cost, and forecasts? Or is the user naming the complete set? Ordinary conversation answers pragmatically. Consequential computation should not always have to guess.

Shambibble's negative prompting can be reread in this light. "No fruit, no sphere, no circle" is not merely stylistic emphasis. It is an attempt to close a region of the output space that the affirmative phrase left open. Patent drafting has a vocabulary for the same underlying act: claim what must be present, decide whether unrecited matter remains permissible, and make closure explicit when it matters.

The lesson is not that future prompts should literally use patent terms. A word such as "comprising" has stable legal force only because an interpretive institution supplies that force. Dropping it into a model prompt does not magically create computational semantics. The lesson is that prompt

[5] 35 U.S.C. § 112(b), https://uscode.house.gov/view.xhtml?edition=prelim&num=0&req=granuleid%3AUSC-prelim-title35-section112.

[6] 572 U.S. 898, 909–11 (2014), https://www.govinfo.gov/app/details/USREPORTS-572/USREPORTS-572-898.

[7] U.S. Patent & Trademark Office, *MPEP* § 2175, https://www.uspto.gov/web/offices/pac/mpep/s2175.html.

[8] U.S. Patent & Trademark Office, *MPEP* § 2111.03, in § 2111, https://www.uspto.gov/web/offices/pac/mpep/s2111.html.

languages need *scope operators*. "Include" and "only" are beginnings; schemas, enumerations, tool contracts, and explicit open/closed flags are stronger forms. Lawyers matter here because they already know that a small lexical transition can silently redraw an entire boundary.

V. Prompt Prosecution

The deepest legal analogy is not between a patent claim and a single prompt. It is between *patent prosecution* and iterative prompting.

A patent application begins with proposed claims. An examiner construes them, applies legal and technical objections, and issues rejections. The applicant argues, narrows, defines, cancels, or rewrites. Those changes do not vanish into the final text. They become prosecution history. In *Festo*, the Supreme Court explained why narrowing amendments can matter later: a patentee may be prevented from recapturing through equivalence territory surrendered to obtain the patent.[9]

The prompt analogue is immediately recognizable. The user drafts an instruction. The system produces an adverse construction. The user adds a limitation. A second failure reveals an interaction. Another limitation is added. Yet most prompt interfaces preserve only a conversational pile or the latest polished version. They do not preserve a jurisprudence of why each constraint acquired force.

Shambibble's empirical method already points toward prosecution history. When users copied a long successful prompt full of unexplained phrases, he objected that they did not know which terms mattered. His proposed remedy was controlled amendment: hold the seed fixed and "take them out one by one." He also refused to claim exact causal knowledge where interactions and word order remained uncertain. The manual, he said, should teach the small number of things supported by simple toy examples.[10]

That is prosecution as experimentation. Each amendment should carry four things: the prior language, the adverse behavior that motivated the change, the reason the amendment was thought to exclude it, and a regression test showing whether it did. The resulting prompt is no longer a polished paragraph. It is a versioned interpretive record.

This record has negative content. A deleted constraint may have been irrelevant, or it may have been deliberately withdrawn after a model change, or it may have been removed accidentally during simplification. Those histories are not equivalent. The field therefore needs a concept analogous in spirit—not legal effect—to prosecution-history estoppel: a way to mark territory intentionally surrendered so later edits do not unknowingly restore a known failure.

VI. The Missing Skilled Reader

Here the analogy reaches its most productive limit. Patent claim construction asks what a person of ordinary skill in the relevant art would understand, informed by the specification and intrinsic record. That institutional fiction gives the law an answer to a question prompt discourse often leaves unstated: *clear to whom?*

There is no stable person of ordinary skill in the prompt art. The interpreter is itself part of the changing technical object. Shambibble noticed this at the token level: a system could expose which words had been joined or split in tokenization. He predicted that much prompt craft might disappear

[9] *Festo Corp. v. Shoketsu Kinzoku Kogyo Kabushiki Co.*, 535 U.S. 722, 733–41 (2002).
[10] *MJ Interview 3.wh_shambibble*, supra note 1, at approximately 46:25–48:36.

as natural-language processing improved enough that a user could simply say what they wanted.[11] Both observations undermine the idea that a prompt has a fixed operational meaning apart from a model and runtime.

The same visible text can behave differently with a new model version, a new system instruction, a different tool schema, a changed context window, or a changed action surface. Prompt interpretation is therefore indexed to an execution environment. If patent law's intrinsic evidence includes the claims, specification, and prosecution history, the closest prompt analogue is a larger control field: user instruction, system instruction, definitions, examples, tool descriptions, schemas, active state, and amendment history.

This is also where lawyers must resist importing their own confidence too far. A judge interprets under publicly contestable doctrine. A model samples behavior from a learned distribution inside a proprietary or rapidly changing runtime. It is not a court, not a reasonable person, and not a stable interpretive community. Legal construction therefore cannot replace empirical testing. It tells us what to look for; execution tells us which constructions the machine actually makes.

That is why Shambibble's phrase "studied ignorance" matters. He was willing to formulate local prompt rules and equally willing to say that they might mean nothing in six months. He distinguished using the system playfully from explaining it empirically, and by 2022 had raised his own standard from one fixed-seed comparison to four repeated runs.[12] The ancient master's legal instinct and experimental instinct are inseparable. Construction without testing becomes scholasticism; testing without construction becomes folklore.

VII. Why Lawyers Could Lead Prompt Language

The argument for lawyers is now clearer, and narrower, than the claim that lawyers write well. Lawyers—especially patent lawyers—are trained to author operative language inside an ecology of interpretation. They think in scope, exceptions, definitions, dependencies, amendments, adverse readers, burdens, records, and consequences. They learn that omission can widen a claim, that a single transition can open or close a set, that a later reference needs an antecedent, that definitions must bind, that changed language needs a reason, and that the drafter's private intention is weak evidence against public text.

Those habits are increasingly relevant as models cease to be answer machines and become agents capable of state change. The prompt writer is no longer merely asking for eloquent completion. The writer may be allocating authority: inspect but do not alter; modify these files but not those; use these sources but do not infer missing facts; prepare an email but do not send it; act if a precondition is met; stop if a condition changes. This is not legal work, but it is boundary work.

The lawyer's leadership should therefore be framed as *interpretive engineering*. Its units are not legalese and boilerplate. They are: (1) claim the intended territory; (2) test the broadest materially plausible constructions; (3) distinguish open from closed scope; (4) bind consequential references; (5) identify the context that controls definitions; (6) amend against demonstrated failures; (7) preserve amendment history; and (8) stop at reasonable certainty rather than fictional completeness.

This also identifies what must *not* transfer. Legal practice can reward strategic ambiguity, prolixity, inaccessible jargon, aggressive issue multiplication, and defensive drafting whose social function is exclusion rather than clarity. A prompt language modeled stylistically on contracts or patent claims would be a disaster for ordinary users. The transferable asset is adversarial method, not legal form.

[11] Id. at approximately 04:29 and 48:53–51:32.

[12] Id. at approximately 14:21 and 43:24–48:36.

The empirical question remains open. Patent attorneys may not outperform programmers, technical writers, designers, or experienced operators on real agent tasks. The interview supplies a mechanism and a striking case, not comparative proof. The right test is direct: give different professional groups the same underspecified consequential agent tasks and measure who detects alternate constructions, scope leakage, reference ambiguity, exception failure, and stale amendment history before execution.

VIII. Deeper Questions for a Law of Prompt Language

If the legal analogy is taken seriously, it generates questions more interesting than whether prompts are "really code."

1. **Who is the skilled reader?** If prompt meaning is interpreter-indexed, should specifications be versioned to a model/runtime rather than treated as freestanding text?
2. **What is intrinsic evidence?** When a prompt is disputed, should its meaning be reconstructed from the user turn alone, or from system instructions, tool contracts, schemas, examples, prior corrections, and state?
3. **What is indefiniteness?** Is a prompt indefinite whenever multiple outputs are possible, or only when ambiguity crosses a consequential boundary?
4. **What is the doctrine of equivalents for prompting?** If an agent reaches the authorized function through an unrecited route, is that faithful execution or unauthorized improvisation?
5. **What is surrendered by amendment?** When a user says "never do X again" after a failure, when should that narrowing expire, and who has authority to broaden it?
6. **What is public notice?** When agent actions affect third parties, should instructions be inspectable enough that affected people can know the boundaries of the agent's delegated authority?
7. **Can scope words become true operators?** What would it take for open, closed, optional, exclusive, contingent, and superseded instructions to have stable runtime semantics rather than pragmatic force?
8. **When should the lawyer stop drafting?** If absolute precision is impossible, what evidence establishes reasonable certainty sufficient to permit execution?

These questions reposition lawyers in the emerging field. They are not latecomers asked to regulate a technology built elsewhere. Some of their oldest techniques concern exactly the new problem: how language acquires operative boundaries when another interpreter will decide what the words allow.

Conclusion: Legal Language Is Prompt Language, But Not Because It Sounds Legal

The strongest lesson from Shambibble is easy to miss because his examples are playful. Orange becomes fruit. An alligator will not stay inside a trench coat. A recurring face is extracted from a workaround. Long prompts accumulate unexplained junk. Seeds stop functioning as the control one imagined. These are not merely quirks of an early image model. They are cases in interpretation under consequence.

Patent law provides an older vocabulary for that condition. Claims define territory rather than merely describe objects. Their meaning is tested from outside. Scope can be open or closed. References need grounding. The specification can govern terms. Amendments leave history. Definiteness is judged against reasonable certainty because complete precision is impossible. A drafter who waits until litigation to explain what the words "really meant" has waited too long.

Prompt language now faces the same structural pressure at machine speed. The mature practitioner will not be the person who knows the longest list of magic words. It will be the person who can anticipate interpretation, expose alternative constructions, decide which ambiguity matters, amend with provenance, and test what the machine actually does. In that sense, the patent lawyer is not merely well prepared for prompt engineering. Patent practice is an existing discipline of the central problem prompt engineering is only beginning to name.

The analogy ends where execution begins. Legal meaning is institutional; model behavior is stochastic. A patent claim is not a prompt, and a neural model is not a court. But the difference strengthens rather than defeats the argument. Prompt practice needs both the lawyer and the experimenter: adversarial construction to imagine how language can go wrong, and empirical execution to discover how this interpreter actually makes it go wrong. Shambibble had both instincts before the field had settled on a name for either. That is what makes him an ancient master.

Authorities and Sources

- 35 U.S.C. § 112.
- *Festo Corp. v. Shoketsu Kinzoku Kogyo Kabushiki Co.*, 535 U.S. 722 (2002).
- *Nautilus, Inc. v. Biosig Instruments, Inc.*, 572 U.S. 898 (2014).
- *White v. Dunbar*, 119 U.S. 47 (1886).
- U.S. Patent & Trademark Office, *Manual of Patent Examining Procedure* §§ 2111, 2111.03, 2173, 2175.
- *MJ Interview 3.wh_shambibble*, Oct. 22, 2022 (automated transcript on file with author).

Appendix A — Assembly and Provenance

This working paper was assembled from a preserved research field containing the 2022 Shambibble interview, earlier transcript-grounded research objects, current prompt-language research, and the patent-law sources identified above. The exact assembly instrument is preserved as `SLIPCASE_FINAL_PROMPT.txt` and under `_PROMPTS/`. The accompanying `SOURCE_MAP` distinguishes transcript evidence, legal authority, and compiler synthesis. Immutable prior zettel payloads were not rewritten; the legal extensions are new dated descendants. Assembly prompt SHA-256: `a5ef0e2e3ea56463e25065f19e7313b4e9c8c569b8cceb1a3c5d9f05bd83598a`.